Introduction to LinkedList

Let's discuss one of the fundamental and most used data structures – the Linked List! This chapter aims to unravel the essence of linked lists, their variations, practical applications, and how to implement them across various programming languages.

Section 1: What is a Linked List?

A linked list is a linear data structure where elements are stored in nodes, and each node points to the next one in the sequence, forming a chain-like structure.

1.1 Node Structure

- **Value:** The data element stored in the node.
- **Next:** A reference to the next node in the list.

1.2 Why Linked Lists?

- **Dynamic Size:** Easily grows and shrinks in size.
- **Efficient Insertions/Deletions:** Quick at the beginning and middle of the list.

Section 2: Types of Linked Lists

In the realm of linked lists, there are several types, each serving a unique purpose.

2.1 Singly Linked List

- **Definition:** Each node points to the next node.
- **Use Case:** A music playlist where each song plays after the previous one.

2.2 Doubly Linked List

- **Definition:** Each node has pointers to both the next and the previous nodes.
- **Use Case:** A web browser's history, enabling forward and backward navigation.

2.3 Circular Linked List

- **Definition:** The last node in the list points back to the first node.
- **Use Case:** A multiplayer board game where play returns to the first player after the last.

Section 3: Practical Implementations

3.1 Linked List in Java

Java offers a LinkedList class in its standard library.

```java
import java.util.LinkedList;

public class Main {
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>();
        list.add("Node1");
        list.add("Node2");
        list.add("Node3");
        System.out.println(list);
    }
}
```

3.2 Linked List in Python3

Python provides a list data type which can be used as a linked list due to its dynamic nature.

```python
class Node:
    def __init__(self, data=None):
        self.data = data
        self.next = None

head = Node("Node1")
head.next = Node("Node2")
head.next.next = Node("Node3")

current = head
while current:
    print(current.data)
    current = current.next
```

3.3 Linked List in C++

In C++, linked lists can be manually created using structures and pointers.

```cpp
#include <iostream>

struct Node {
    std::string data;
    Node* next;
};

int main() {
    Node* head = new Node{"Node1"};
    head->next = new Node{"Node2"};
    head->next->next = new Node{"Node3"};

    Node* current = head;
    while (current) {
        std::cout << current->data << std::endl;
        current = current->next;
    }
}
```

3.4 Linked List in JavaScript

In JavaScript, we can create a linked list using objects and references.

```javascript
class Node {
    constructor(data) {
        this.data = data;
        this.next = null;
    }
}

const head = new Node("Node1");
head.next = new Node("Node2");
head.next.next = new Node("Node3");

let current = head;
while (current) {
    console.log(current.data);
    current = current.next;
}
```

← Previous
Solution: Max Of All Subarrays Of Size 'K'

Next →
Operations On Singly Linked List

☐ Mark as Completed